

Gradient-Free Optimisation Introduction and Nelder-Mead

ENEL445 Applied Engineering Optimisation

2025

Plan for the rest of the course

Week 8 (this week)

- Gradient-free Intro, Nelder-Mead
- GPS and DIRECT
- Nature-Inspired algorithms

Week 9

- Linear Programming
- Discrete optimisation, Assignment problems
- Simulated annealing

Week 10

- Game theory
- Surrogate-based optimisation, Maximum likelihood
- Le

Plan for the rest of the course

Week 11

- Jeremy - Power systems optimisation 1
- Jeremy - Power systems optimisation 2
- Our research

Week 12

- Your research
- Your research
- Exam revision

Exam

2Q Joe

1Q Le

1Q Jeremy

Outline

- Gradient-Free Motivation
- Exhaustive Search
- Nelder-Mead Algorithm

Why Gradient-free

- Gradient-based algorithms are efficient in finding local minima for continuous and smooth functions in high dimensional spaces.

However:

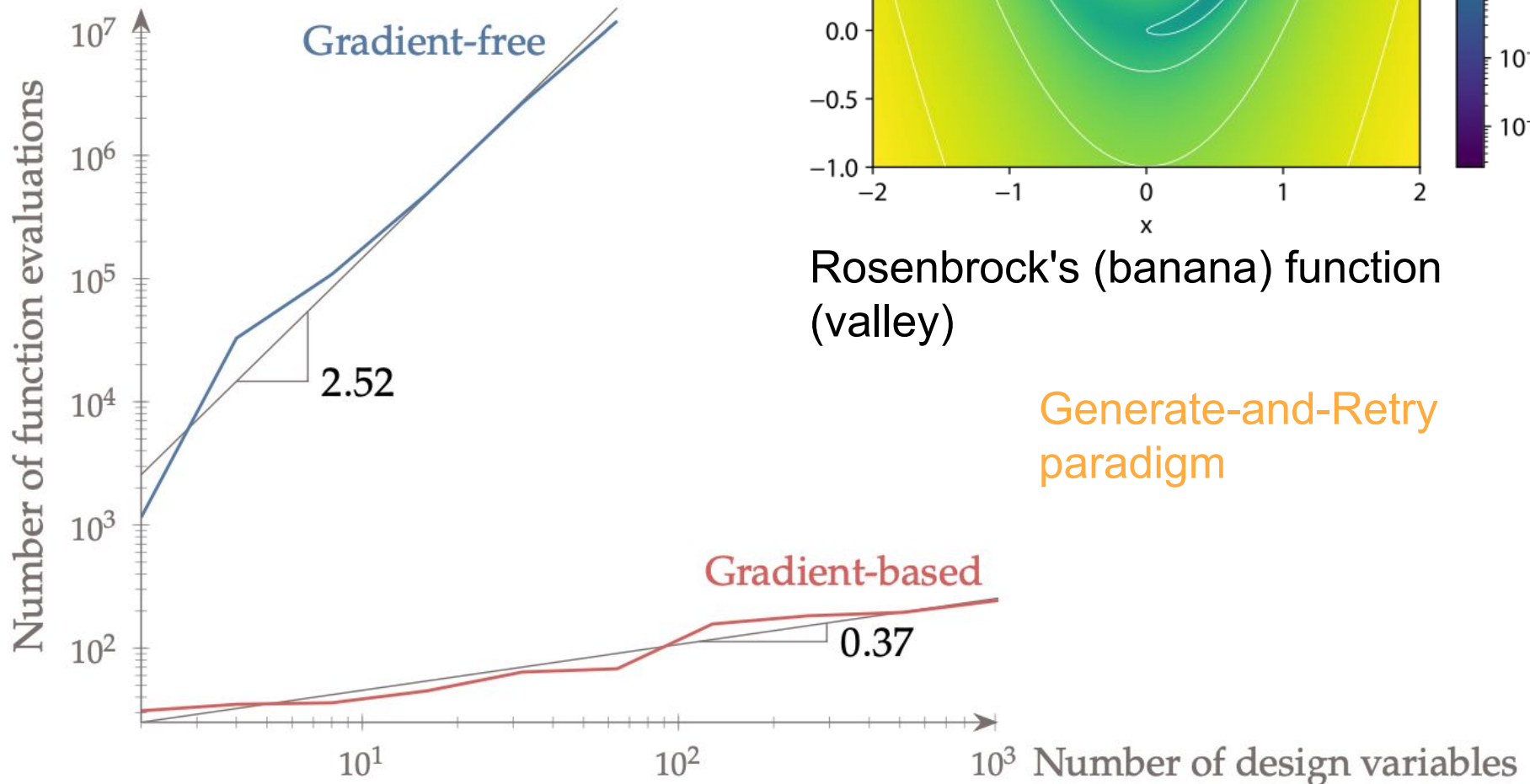
- Objective function may not be smooth.
- Gradients of objective functions may be hard to calculate.
- Many problems have only a relatively small number of dimensions.

Why Gradient-free

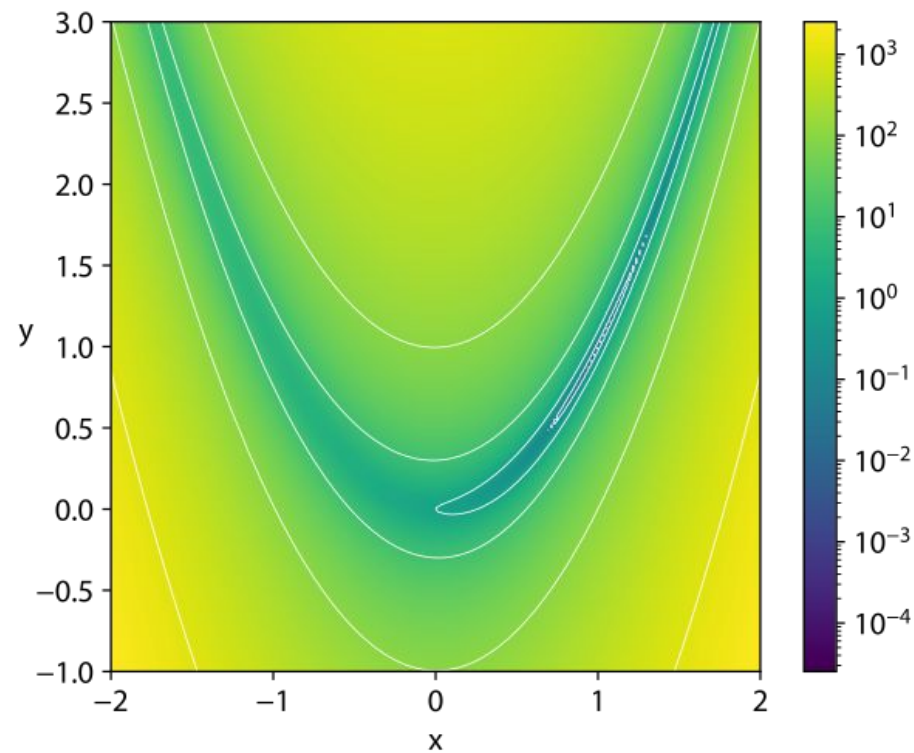
- For **gradient-based algorithms**, function smoothness is essential when deriving the optimality conditions. The Karush–Kuhn–Tucker (KKT) conditions require the function be continuous in **value** (C^0), **gradient** (C^1), and **Hessian** (C^2) in at least a small neighborhood around the optimum.
- When the objective and constraint functions are not smooth enough or when it is not possible to compute derivatives with enough precision, gradient-free algorithms are useful.
- For functions with excessive numerical noise and discontinuities, gradient-free algorithms might be the only option.
- **Gradient-free algorithms** only require evaluation of objective function values. Also known as **zeroth-order** algorithms.

Why Gradient-free

- However, the cost of gradient-free optimisation is sensitive to the cost of the function evaluations



Aside: Test Functions



Rosenbrock's (banana) function (valley)

$$f(\mathbf{x}) = \sum_{i=1}^{d-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$$

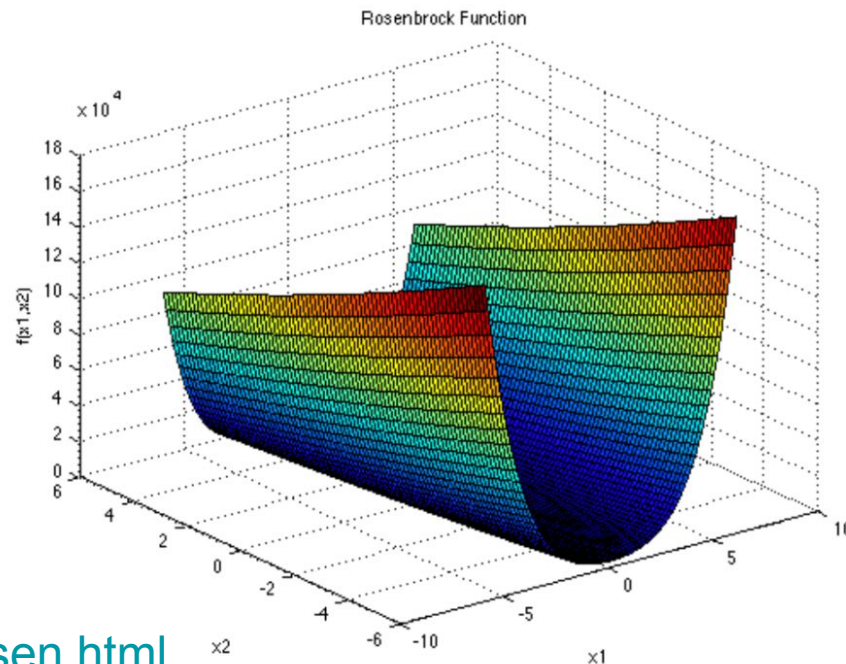
$$f(\mathbf{x}^*) = 0, \text{ at } \mathbf{x}^* = (1, \dots, 1)$$

Aside: Test Functions

Virtual Library of Simulation Experiments: Test Functions and Datasets

Optimization Test Problems

ROSENBROCK FUNCTION



Aside: Test Functions

HOME

OPTIMIZATION

EMULATION/
PREDICTION

UNCERTAINTY
QUANTIFICATION

MULTI FIDELITY
SIMULATION

CALIBRATION/ TUNING

SCREENING

INTEGRATION

FUNCTIONAL DATA

ABOUT

OTHER TEST
FUNCTIONS AND CODE

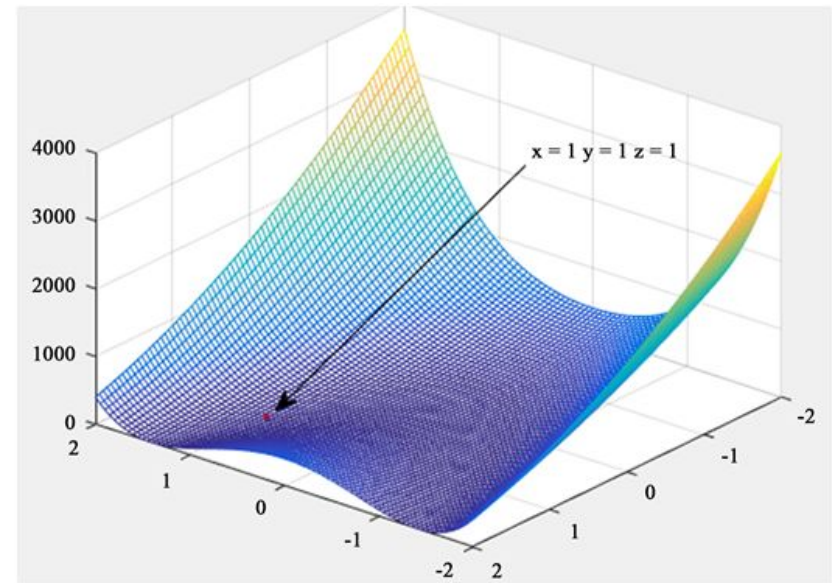
Optimization Test Problems

The functions listed below are some of the common functions and datasets used for testing optimization algorithms. They are grouped according to similarities in their significant physical properties and shapes. Each page contains information about the corresponding function or dataset, as well as MATLAB and R implementations.

Many Local Minima

1. [Ackley Function](#)
2. [Bukin Function N. 6](#)
3. [Cross-in-Tray Function](#)
4. [Drop-Wave Function](#)
5. [Eggholder Function](#)
6. [Gramacy & Lee \(2012\) Function](#)
7. [Griewank Function](#)
8. [Holder Table Function](#)
9. [Langermann Function](#)
10. [Levy Function](#)
11. [Levy Function N. 13](#)
12. [Rastrigin Function](#)
13. [Schaffer Function N. 2](#)
14. [Schaffer Function N. 4](#)
15. [Schwefel Function](#)
16. [Shubert Function](#)

Bowl-Shaped



<https://www.sfu.ca/~ssurjano/optimization.html>

Local vs Global

- Some gradient-free methods feature a global search that increases the likelihood of finding the global minimum. This feature makes gradient-free methods a common choice for multimodal problems.
- However, not all gradient-free methods are global search methods; some perform only a local search. Conversely, even though gradient-based methods are by themselves local search methods, they are often combined with global search strategies.
- It is not necessarily true that a global search, gradient-free method is more likely to find a global optimum than a multistart gradient-based method.
- As always, **problem-specific testing** is needed.

No Free Lunch Theorem Wolpert & Macready (1997)

- “Any two optimization algorithms are equivalent when their performance is averaged across all possible problems.”
- Each "restaurant" (problem-solving procedure) has a "lunch plate" (problem) with a "price" (the performance of the procedure in solving the problem). The menus of restaurants are identical except in one regard – the prices are shuffled from one restaurant to the next. For an omnivore who is as likely to order each plate as any other, the average cost of lunch does not depend on the choice of restaurant. But a vegan or a carnivore who seeks economy might pay a high average cost for lunch. To methodically reduce the average cost, one must use advance knowledge of a) what one will order and b) what the order will cost at various restaurants. That is, improvement of performance in problem-solving hinges on using prior information to match procedures to problems.

No Free Lunch Theorem

Wolpert, D. H. & Macready, W. G. (1997). "No Free Lunch Theorems for Optimization". IEEE Transactions on Evolutionary Computation. 1: 67–82.

Wolpert, David (1996). "The Lack of A Priori Distinctions between Learning Algorithms". Neural Computation. Vol. 8. pp. 1341–1390.

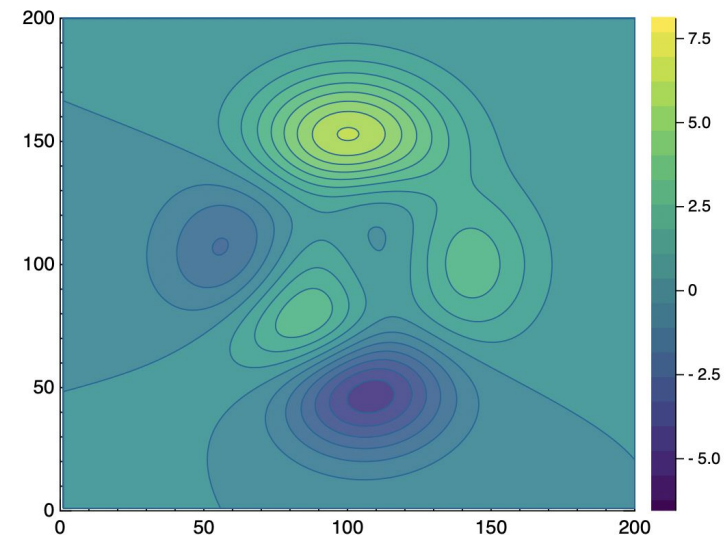
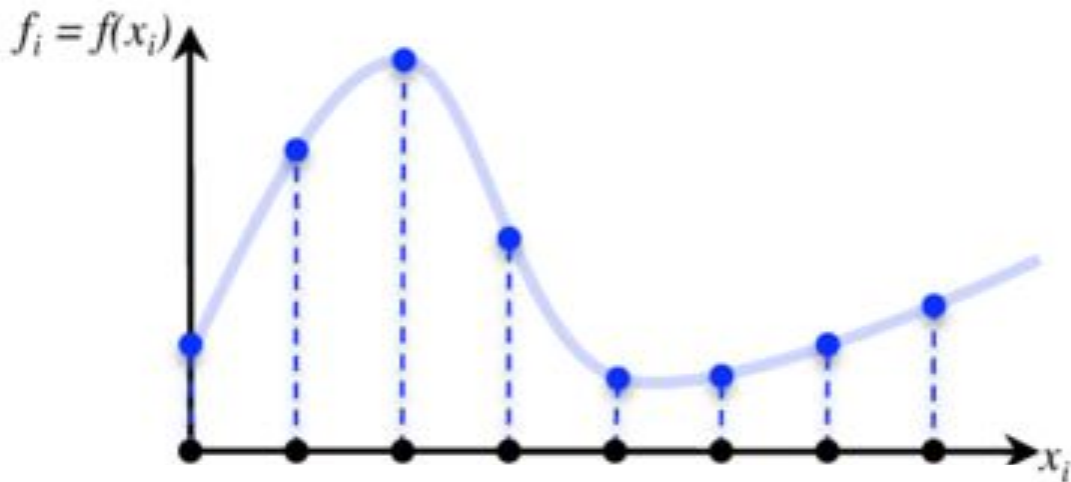
Schaffer, Cullen (1994). "A conservation law for generalization performance". In Willian, H.; Cohen, W. (eds.). International Conference on Machine Learning. San Francisco: Morgan Kaufmann. pp. 259–265.

	Search		Algorithm		Function evaluation		Stochasticity	
	Local	Global	Mathematical	Heuristic	Direct	Surrogate	Deterministic	Stochastic
Nelder–Mead	•			•	•		•	
GPS		•	•		•		•	
MADS		•	•		•			•
Trust region	•		•			•	•	
Implicit filtering	•		•			•	•	
DIRECT		•	•		•		•	
MCS		•	•		•		•	
EGO		•	•			•	•	
Hit and run		•		•	•			•
Evolutionary		•		•	•			•
Simulated Annealing		•		•	•			•

Exhaustive Search

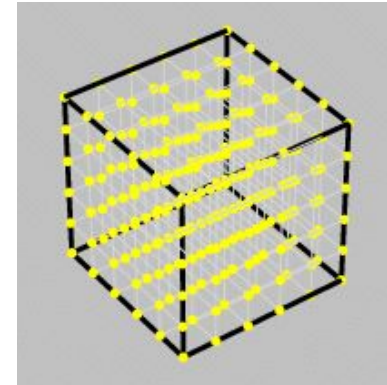
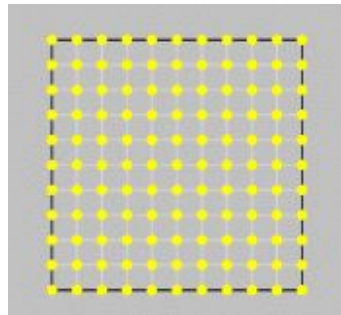
Exhaustive Search

- Also called brute-force search
- Discretise the search space (grid)
- Systematically check all possible points
- Advantages:
 - Easy to implement
 - Embarrassingly parallelisable

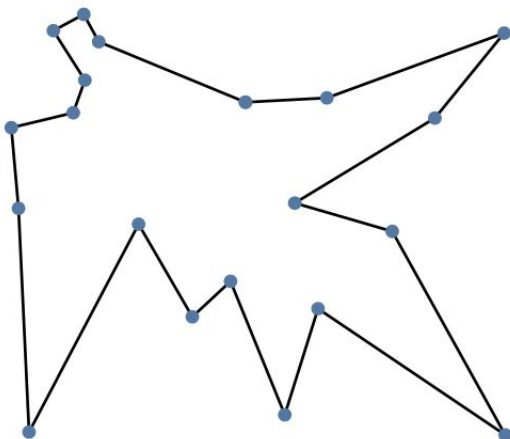


Exhaustive Search

- Disadvantage: Performance of the algorithm scales exponentially (or worse) with the number of design variables



- Example: TSP with 20 cities: $(20-1)! = 19!$ possibilities



```
[In [1]: import numpy as np
```

```
[In [2]: np.math.factorial(19)
```

```
Out[2]: 121645100408832000
```

```
[In [3]: np.math.factorial(19) / 1e9 / (60*60*24*365)
```

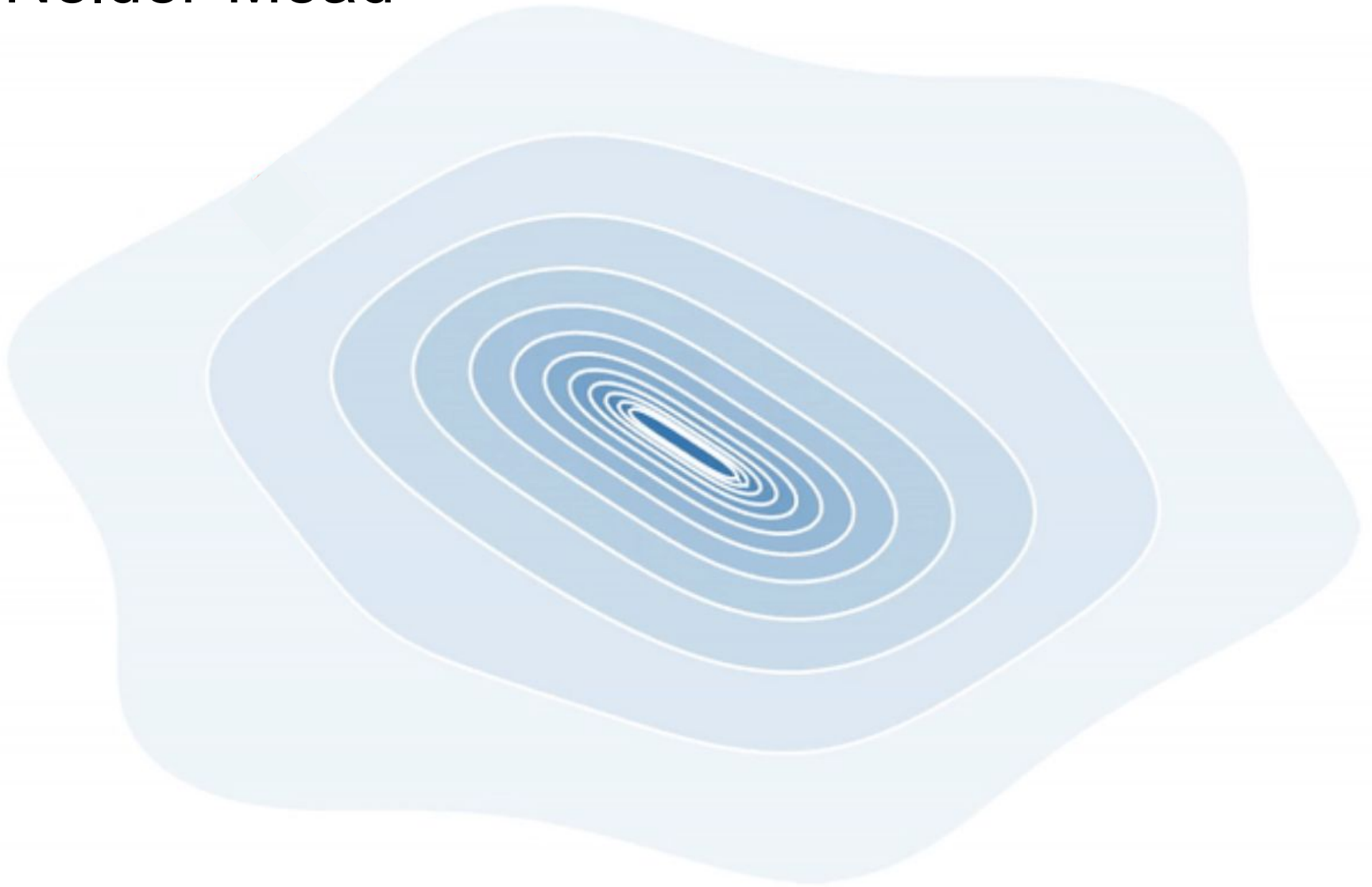
```
Out[3]: 3.8573408298082192
```

```
[In [4]: np.math.factorial(19) / 3e9 / (60*60*24)/28
```

```
Out[4]: 16.76106432
```

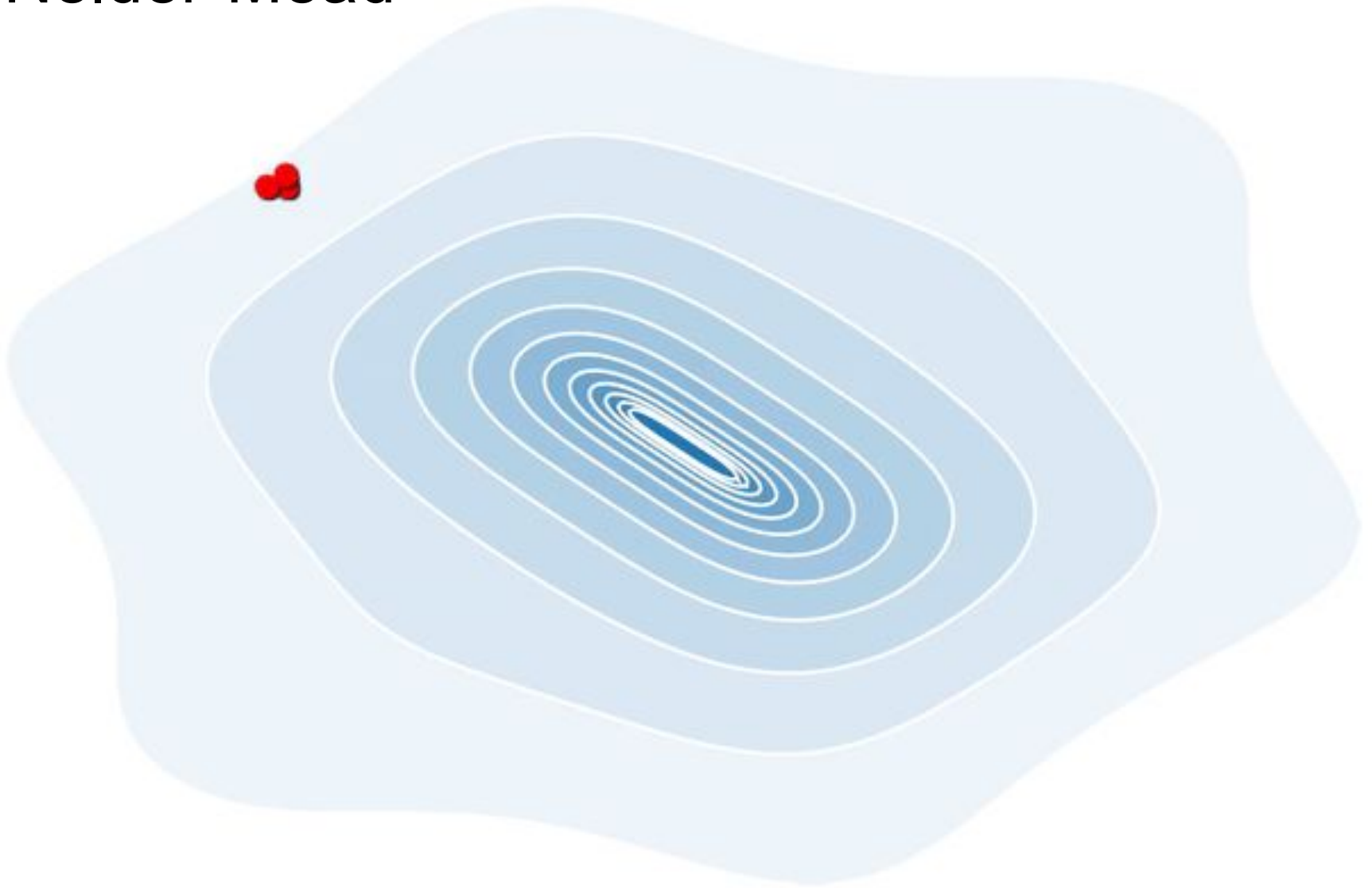
Nelder-Mead

Nelder-Mead



<https://www.npmjs.com/package/@nwaltham/fmin>

Nelder-Mead

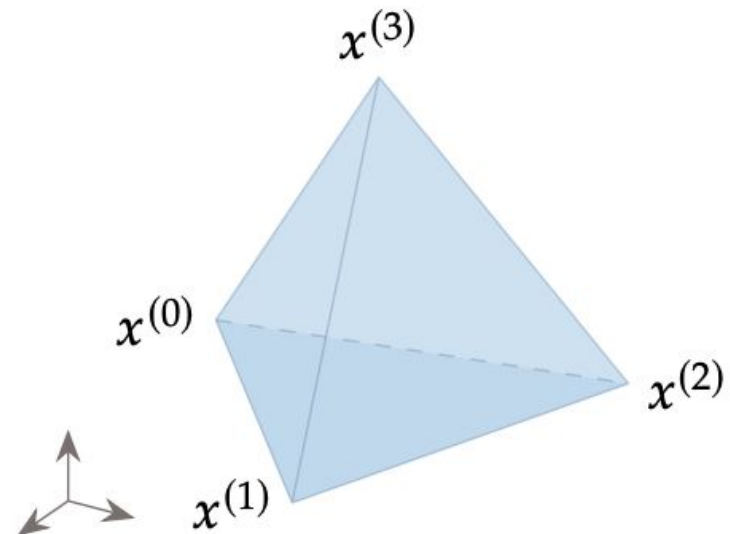
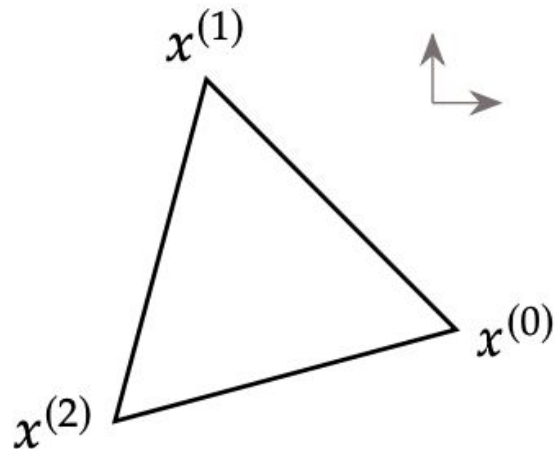


<https://www.npmjs.com/package/@nwaltham/fmin>

Nelder-Mead

Nelder and Mead, “A simplex method for function minimization”, 1965.

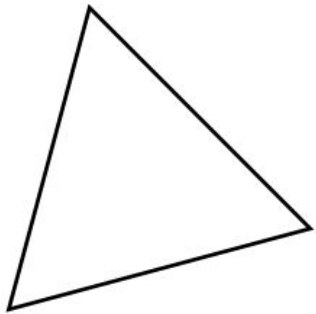
- The Nelder-Mead algorithm is based on a **simplex**, which is a geometric shape defined by a set of $n+1$ points in the design space of n variables
- In two-dimensions, the simplex is a triangle, and in three-dimensions, it becomes a tetrahedron



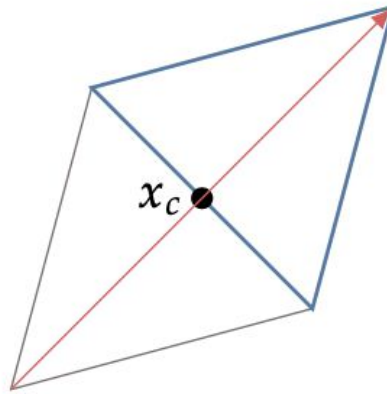
Nelder-Mead

- The objective f is evaluated for every point, and the points are ordered based on the respective values of f , from the lowest to the highest, giving an ordered list of simplex points $X = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n-1)}, \mathbf{x}^{(n)}\}$
- Each iteration aims to replace the worst point with a better one to form a new simplex
- The algorithm modifies the simplex at each iteration using five simple operations

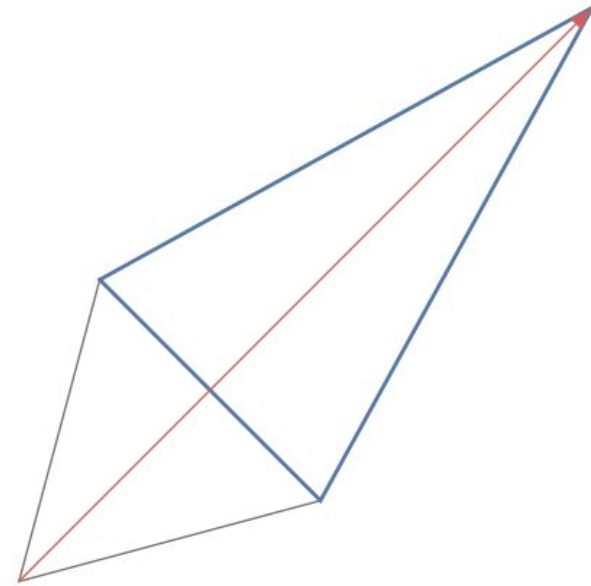
Nelder-Mead



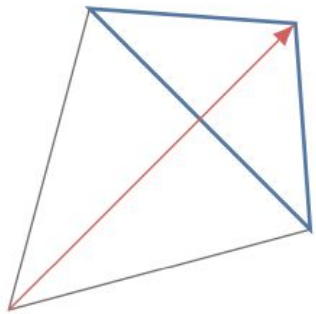
Initial simplex



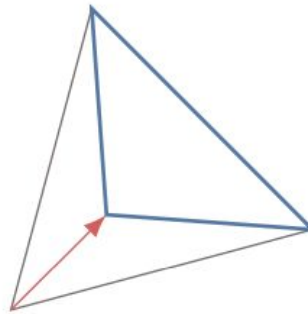
Reflection ($\alpha = 1$)



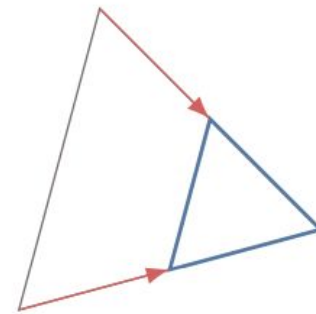
Expansion ($\alpha = 2$)



Outside contraction
($\alpha = 0.5$)

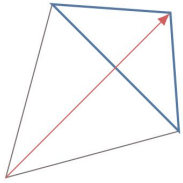


Inside contraction
($\alpha = -0.5$)

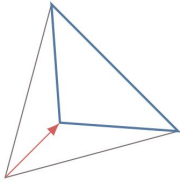


Shrink

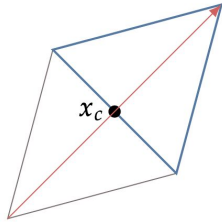
Nelder-Mead



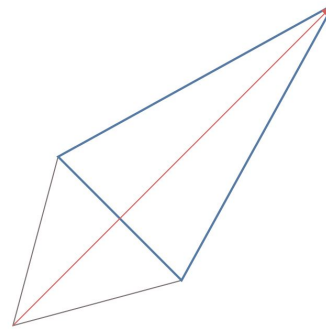
Outside contraction
($\alpha = 0.5$)



Inside contraction
($\alpha = -0.5$)



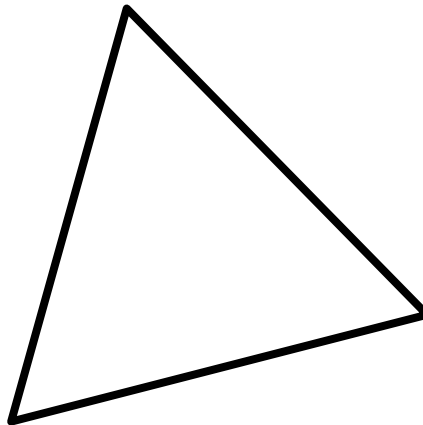
Reflection ($\alpha = 1$)



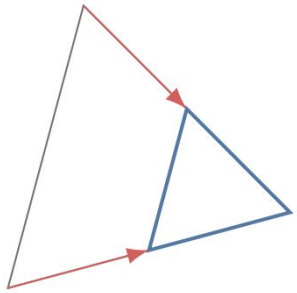
Expansion ($\alpha = 2$)

$$x_c = \frac{1}{n} \sum_{i=0}^{n-1} x^{(i)}$$

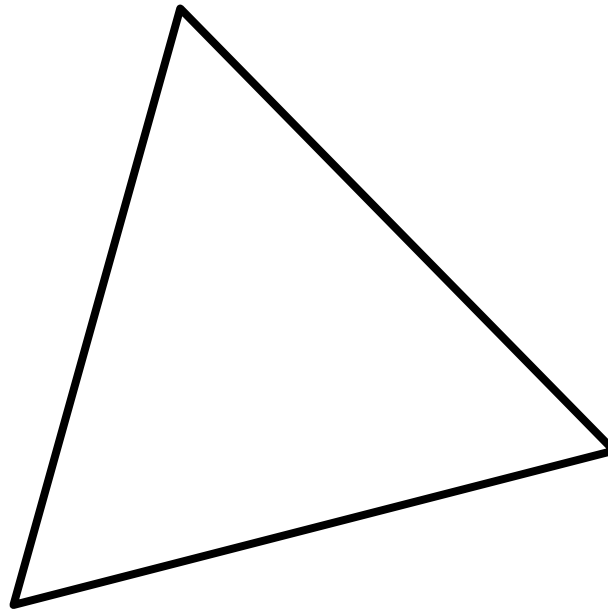
$$x = x_c + \alpha (x_c - x^{(n)})$$



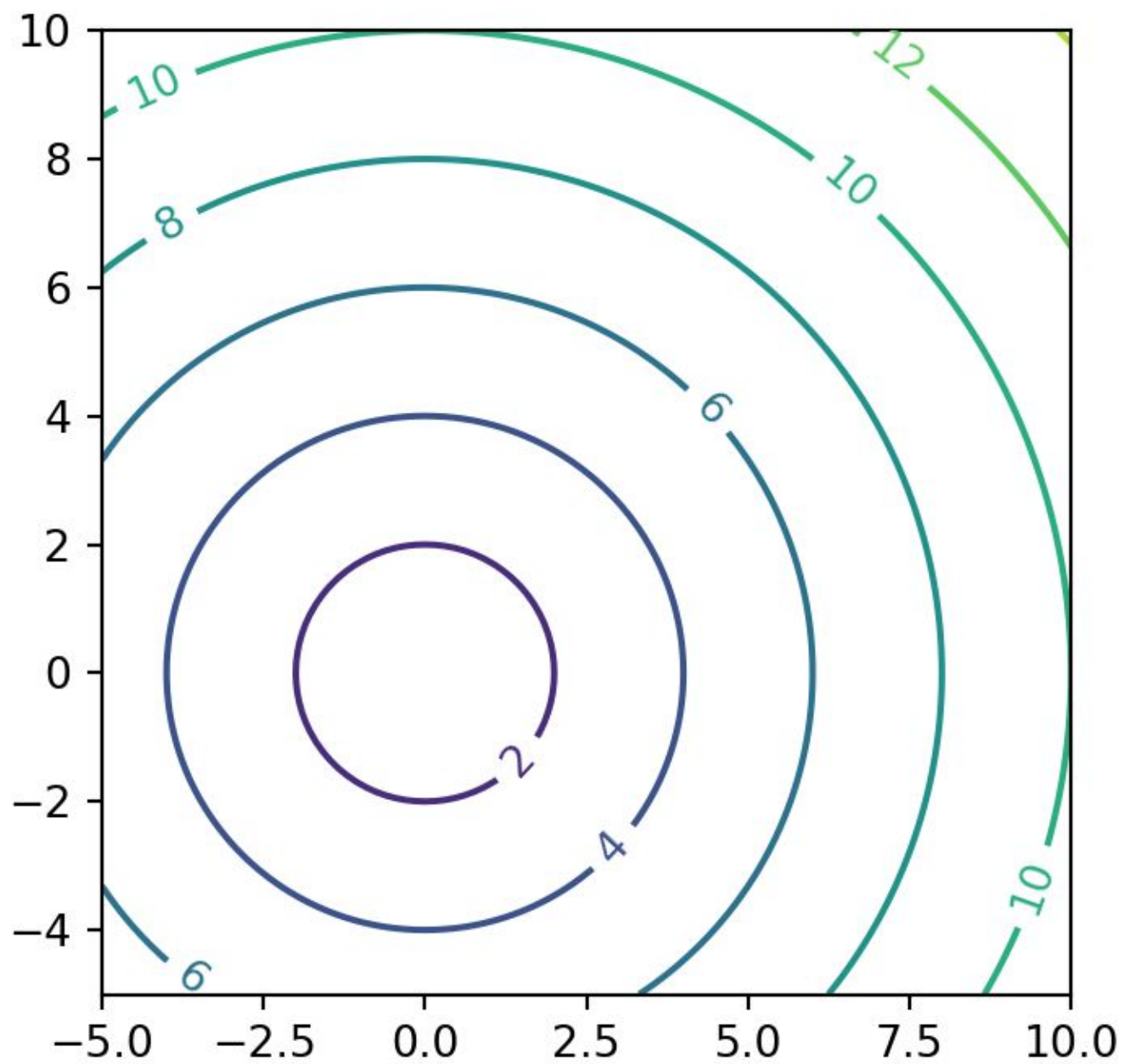
Nelder-Mead

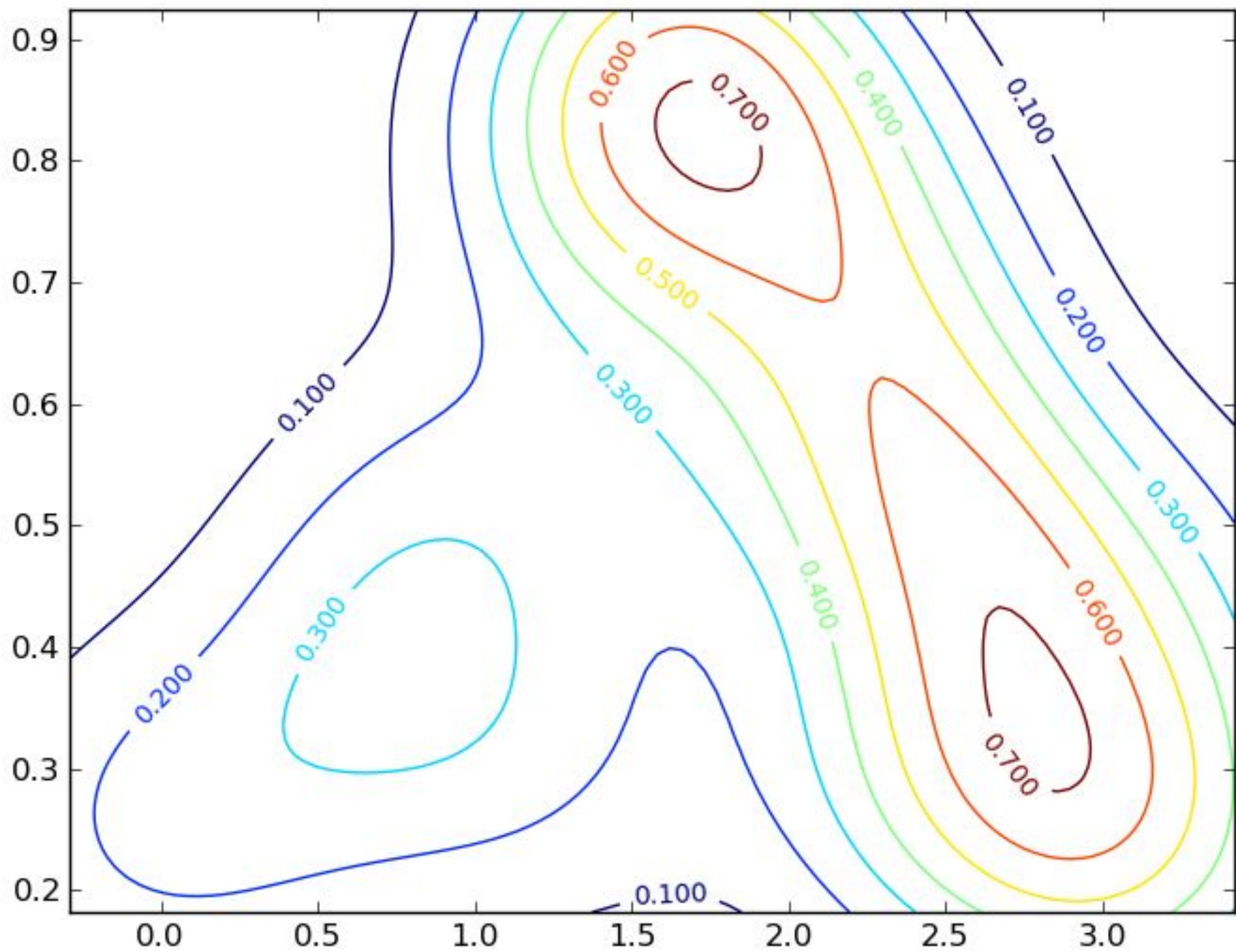


Shrink



$$x^{(i)} = x^{(0)} + \gamma \left(x^{(i)} - x^{(0)} \right) \quad \text{for } i = 1, \dots, n$$





Nelder-Mead - Convergence Metrics

- Size of the simplex:

$$\Delta_x = \sum_{i=0}^{n-1} \|x^{(i)} - x^{(n)}\|$$

- Standard deviation of objective function values:

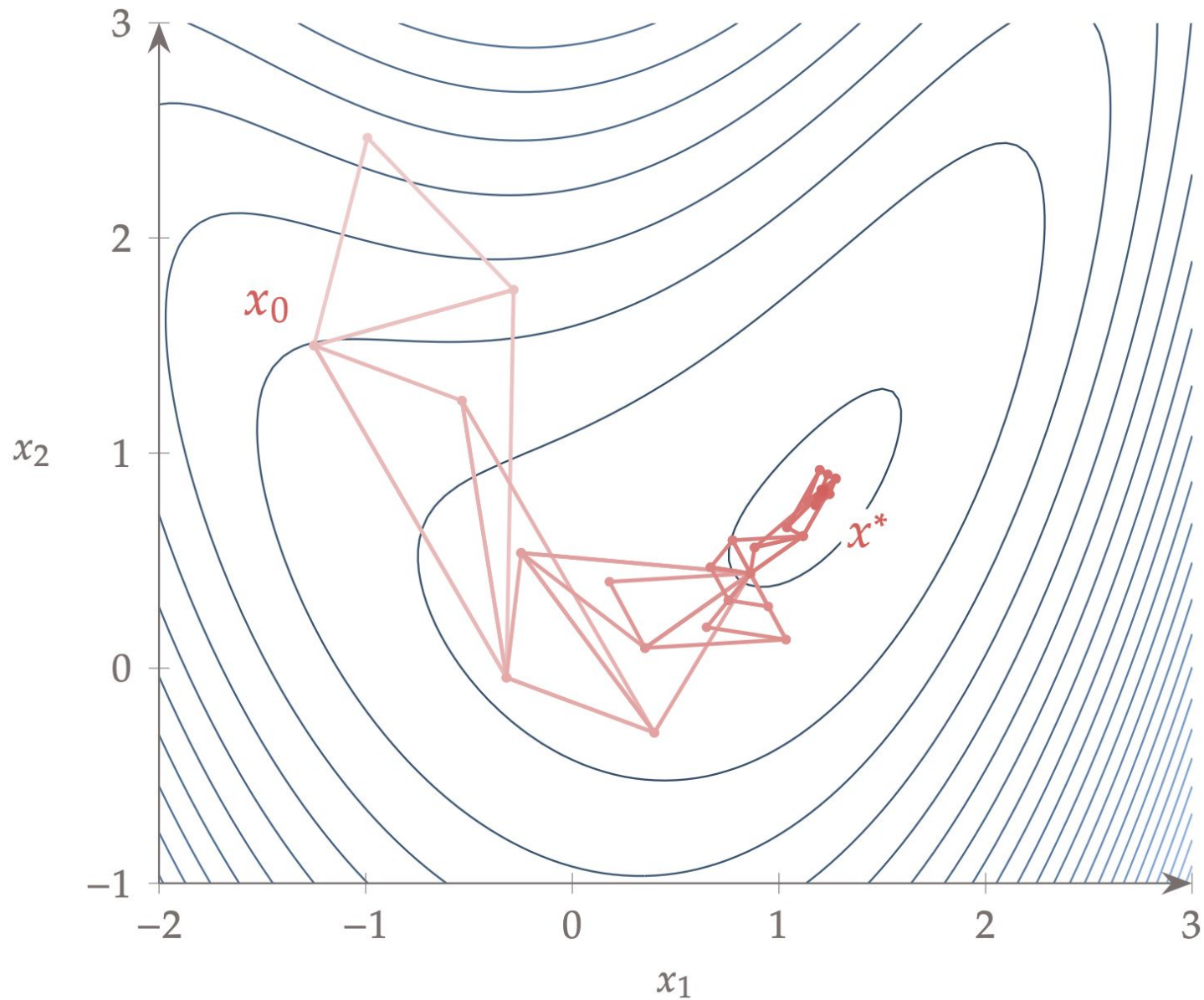
$$\Delta_f = \sqrt{\frac{\sum_{i=0}^n \left(f^{(i)} - \bar{f}\right)^2}{n+1}}$$

- Difference between the best and worst values in the simplex.

Nelder-Mead can converge to non-stationary points - so check your results! [1]

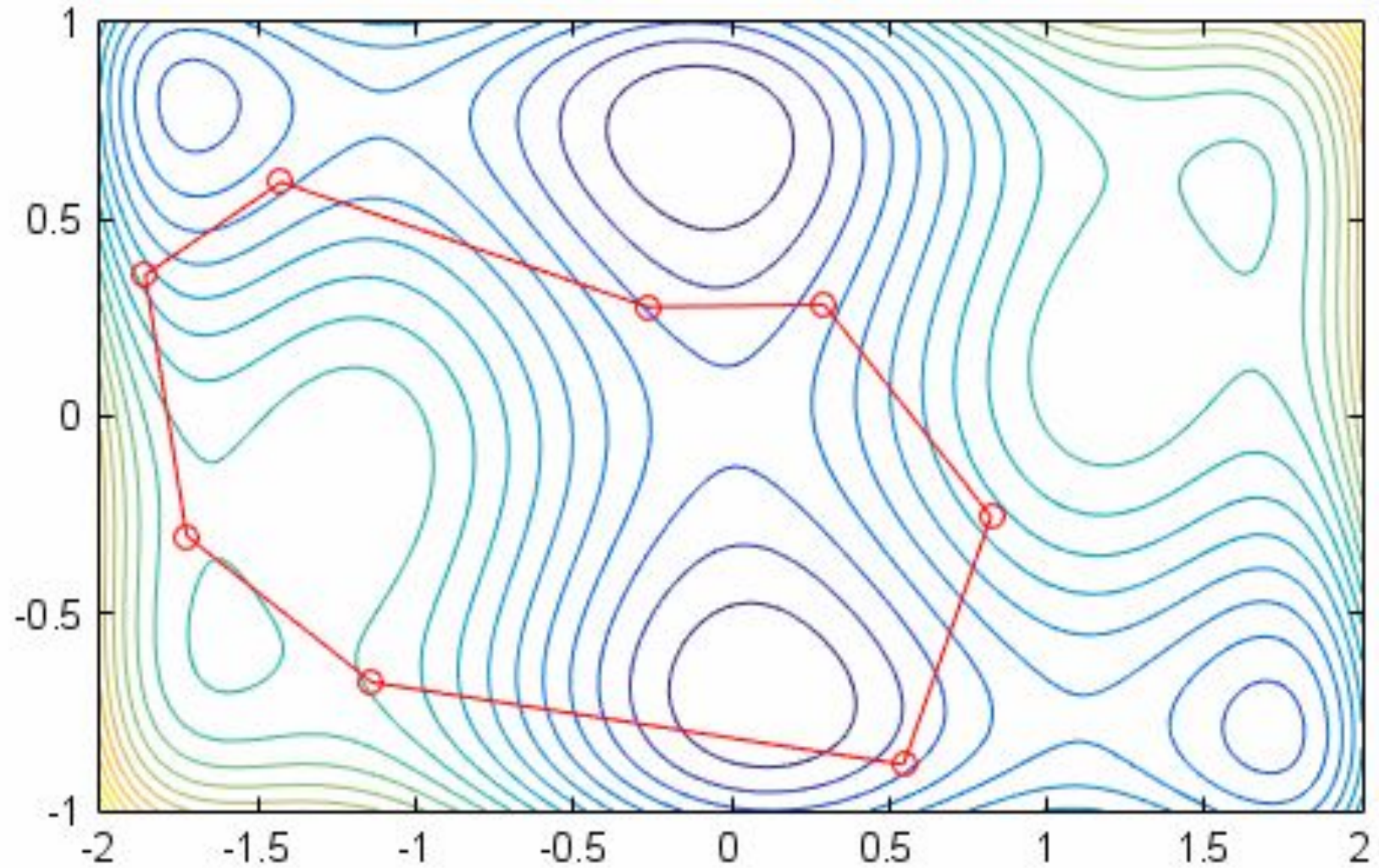
[1] McKinnon (1996), “Convergence of the Nelder-Mead simplex method to a non-stationary point”

Nelder-Mead - 2D



Nelder-Mead - 1D

Nelder-Mead - Extension



Section Navigation

scipy

scipy.cluster

scipy.constants

scipy.datasets

scipy.differentiate

scipy.fft

scipy.fftpack

scipy.integrate

scipy.interpolate

scipy.io

scipy.linalg

scipy.ndimage

scipy.odr

scipy.optimize

scipy.signal

scipy.sparse

scipy.spatial

scipy.special

 > SciPy API > Optimization and root finding ([scipy.optimize](#)) > minimize(met...

minimize(method='Nelder-Mead')

minimize(*fun*, *x0*, *args=()*, *method=None*, *jac=None*, *hess=None*,
hessp=None, *bounds=None*, *constraints=()*, *tol=None*, *callback=None*,
options=None)

Minimization of scalar function of one or more variables using the Nelder-Mead algorithm.

See also

For documentation for the rest of the parameters, see

[scipy.optimize.minimize](#)

Options:

disp : *bool*

Set to True to print convergence messages.

maxiter, **maxfev** : *int*

Summary

- Gradient-free algorithms only require the evaluation of function values.
- Useful when the objective and constraint functions are not smooth enough or when it is not possible to compute derivatives with enough precision.
- Often easier to implement.
- However, the overall cost of gradient-free optimisation is sensitive to the cost of the function evaluations because they require many iterations for convergence, and the number of iterations scales poorly with the number of design variables.
- The Nelder-Mead algorithm is based on a simplex

Exercises

- List the pros and cons of gradient-based vs gradient-free optimisation methods
- Write out the key equations for the Nelder-Mead method and understand what each equation means
- Find a 2D contour map, choose a starting point, draw out one iteration of Nelder-Mead



Thanks!